



Abstraction

Lesson Objectives



At the end of this lesson, you should be able to:

- Describe the concept of abstraction
- Explain the importance of abstraction

Topic Outline



What is Abstraction?



Why is Abstraction Important?



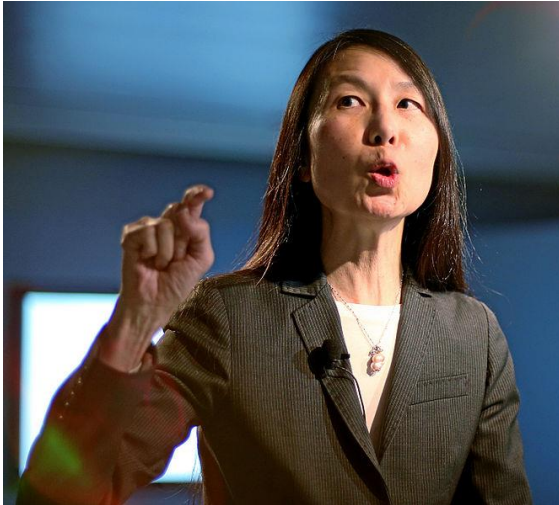
Algorithm Abstraction and Data Abstraction

What is Abstraction?

Abstraction

- **simplifies** things
- identifies what is **important** without worrying too much about the detail
- allows us to **manage complexity**

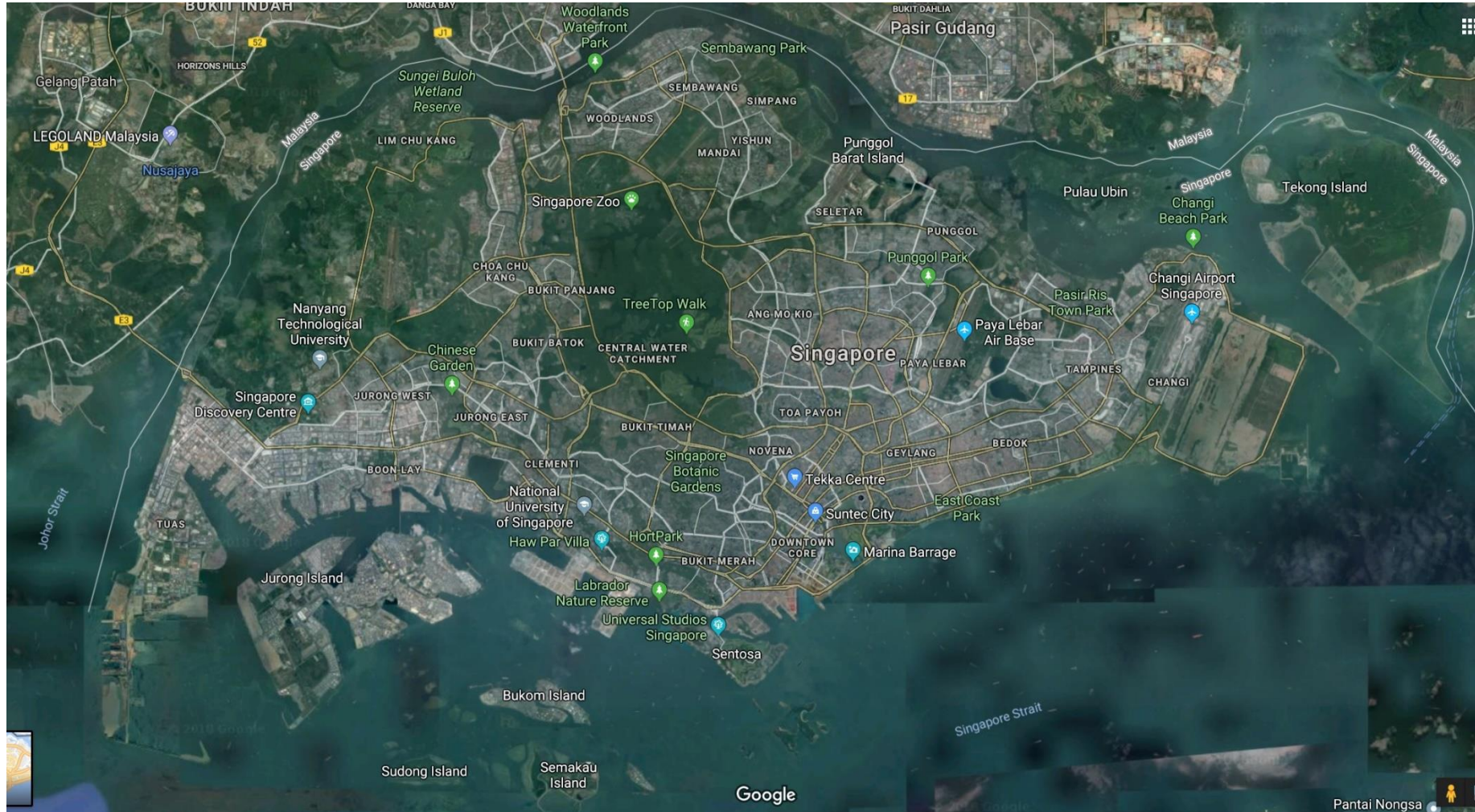
What is Abstraction? (Cont'd)



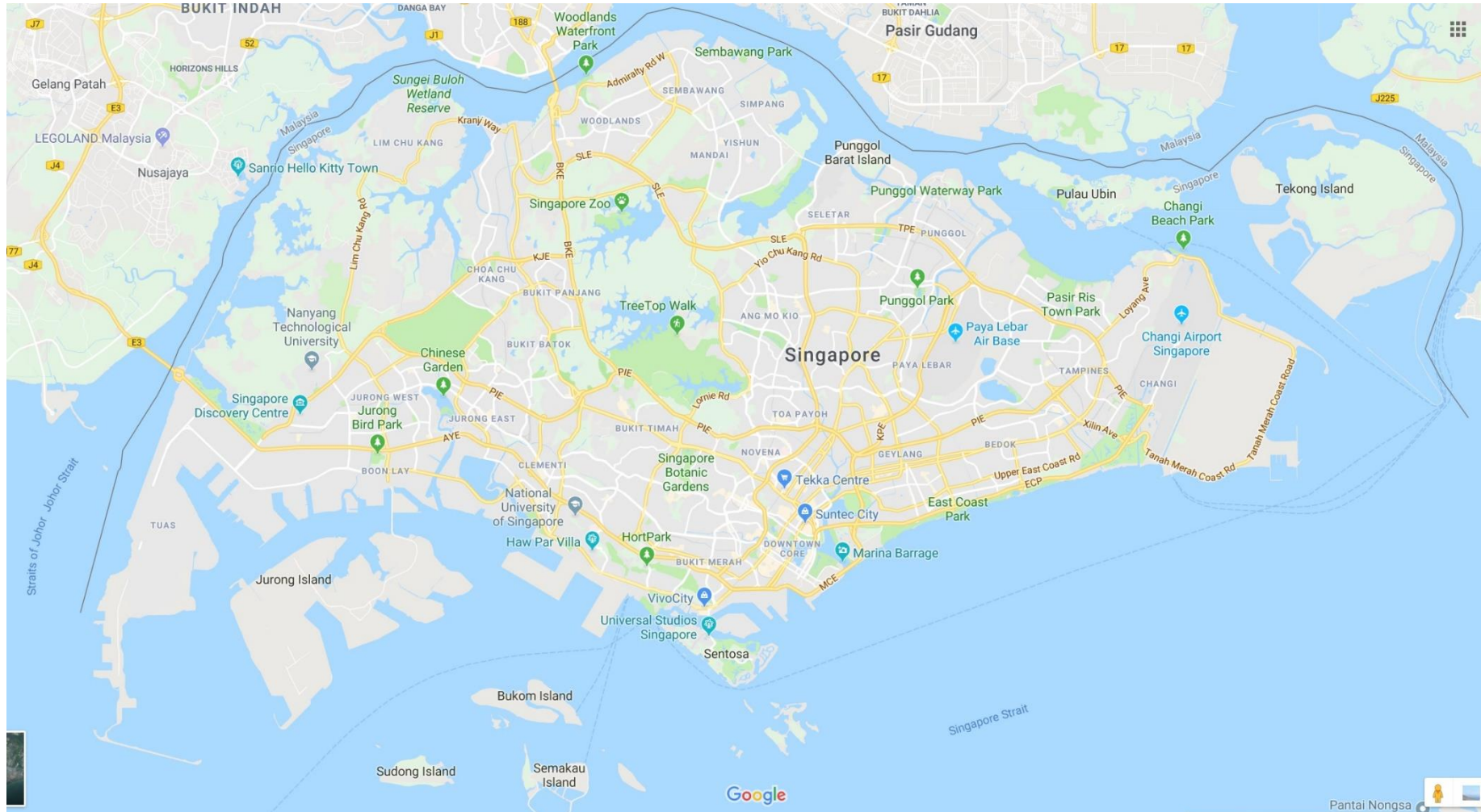
“The abstraction process - deciding what details we need to highlight and what details we can ignore - underlies computational thinking”.

Jeannette Wing, 2008

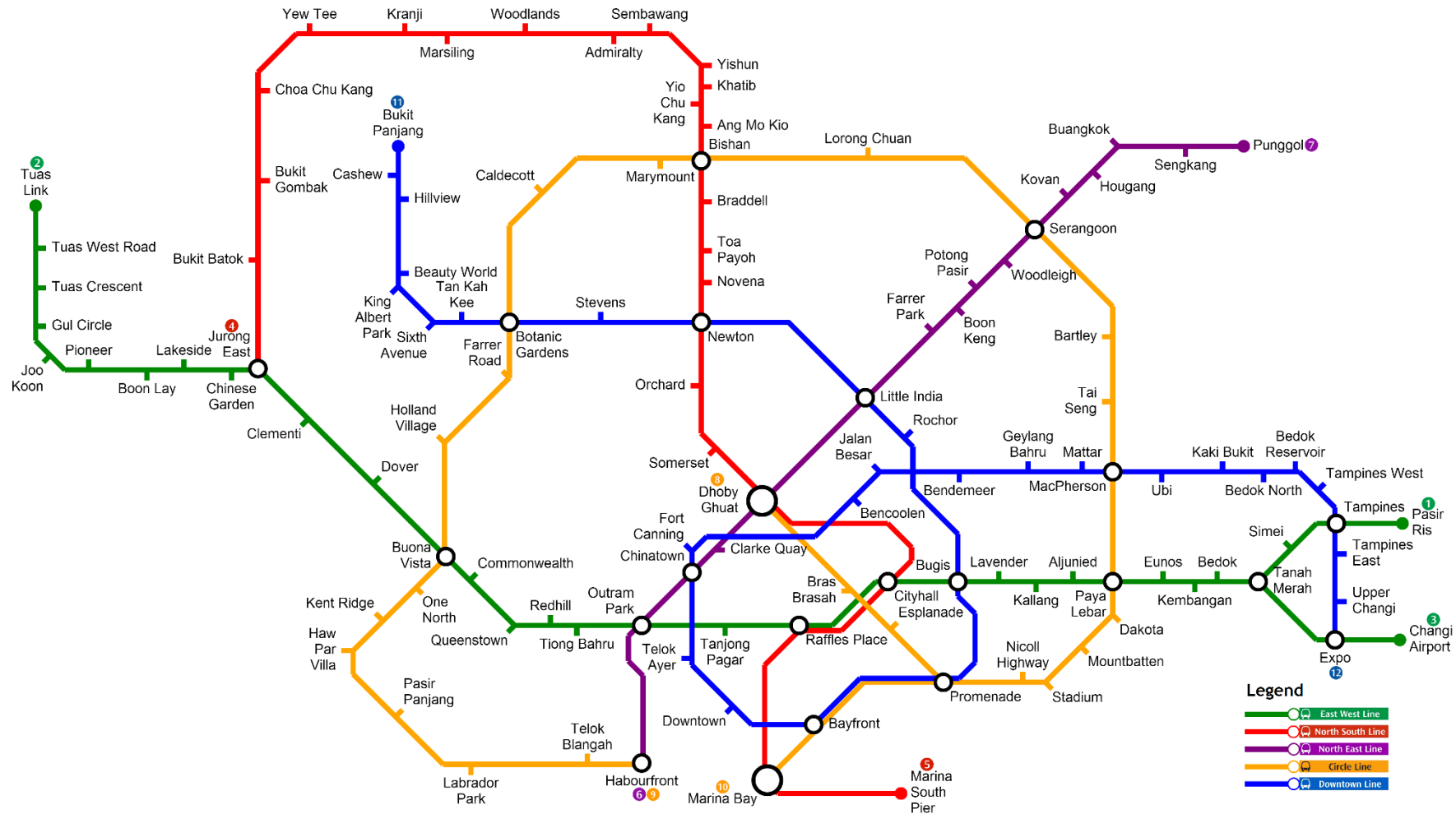
Abstraction Example



Abstraction Example (Cont'd)



Abstraction Example (Cont'd)

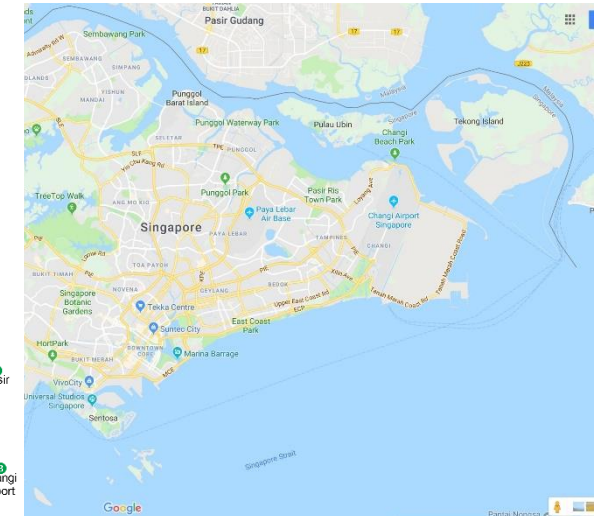
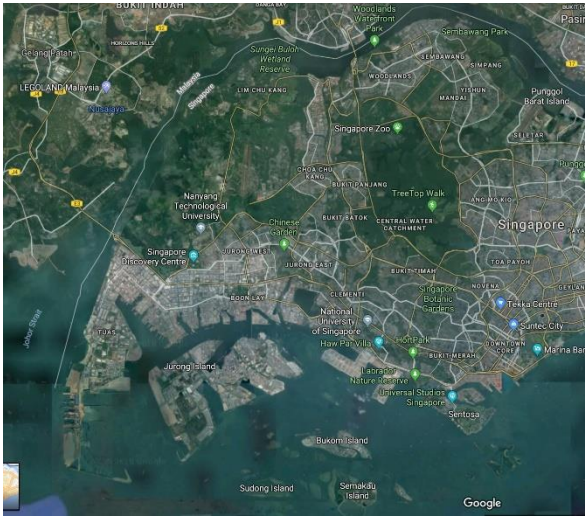


Why is Abstraction Important?

- A key element of computing is the complexity of the systems we build.
- **Abstraction** provides a means to distil what is essential, giving a manageable approach to create computational solutions.
- **Abstractions** are sometimes represented as **layers** or **hierarchies**, allowing us to view things at different degrees of detail.

How many MRT stations are there in Singapore?

Which abstraction do you want to use?



Abstraction in Different Aspects

Abstraction in Data: **Data Structures**
(Strings, Lists, Tuples, Dictionaries, etc.)



Programs = Algorithms + Data Structures

(1976, Niklaus Wirth)

Abstraction in Algorithms:
Functions

In this lesson, we have learned:

- What Abstraction is
- The Importance of Abstraction

References for Images

No.	Slide No.	Image	Reference
1	6		By World Economic Forum - Flickr: An insight, an idea: Jeannette Wing, CC BY-SA 2.0, retrieved June 25, 2018 from https://commons.wikimedia.org/w/index.php?curid=24229759
2	7, 11		Map of Singapore. Retrieved June 22, 2018 from Google Earth. US Dept of State Geographer Data SIO, NOAA, U.S. Navy, NGA GEBCO ©2018 Google Image Landsat/ Copernicus.
3	8, 11		Map of Singapore. Retrieved June 22, 2018 from Google Maps. Map data ©2018 Google.
4	9, 11		By Josephsolomon92 - Own work, CC BY-SA 3.0, retrieved June 25, 2018 from https://en.wikipedia.org/w/index.php?curid=41463804 .
5	12		By Tyomitch - Own work, Copyrighted free use, retrieved June 25, 2018 from https://commons.wikimedia.org/w/index.php?curid=449735 .

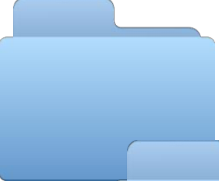


Composite Types in Python



At the end of this lesson, you should be able to:

- Discuss the concept of composite types
- Explain the importance of composite types
- Use composite types in Python to solve problems



What are Composite Types/ Data Structure?



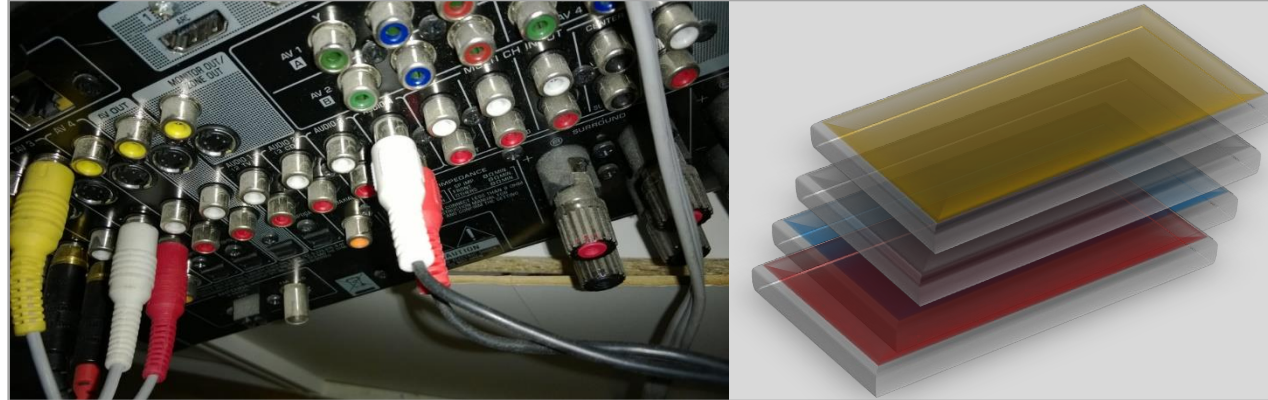
Why are Composite Types/ Data Structure Important?



**Three Common Data Structures in Python
(and Their Operations):**

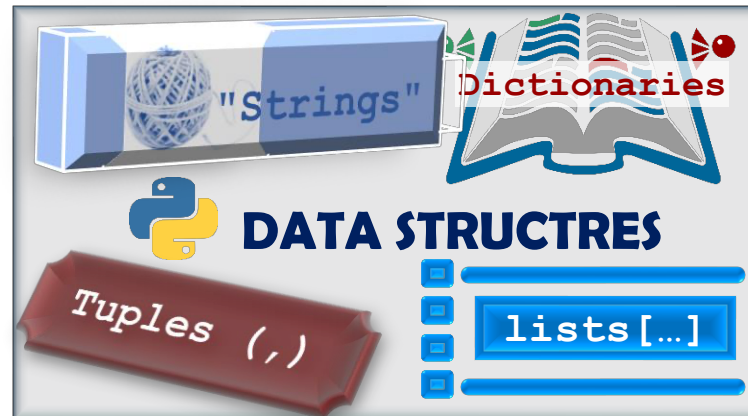
- **List**
- **Tuples**
- **Dictionaries**

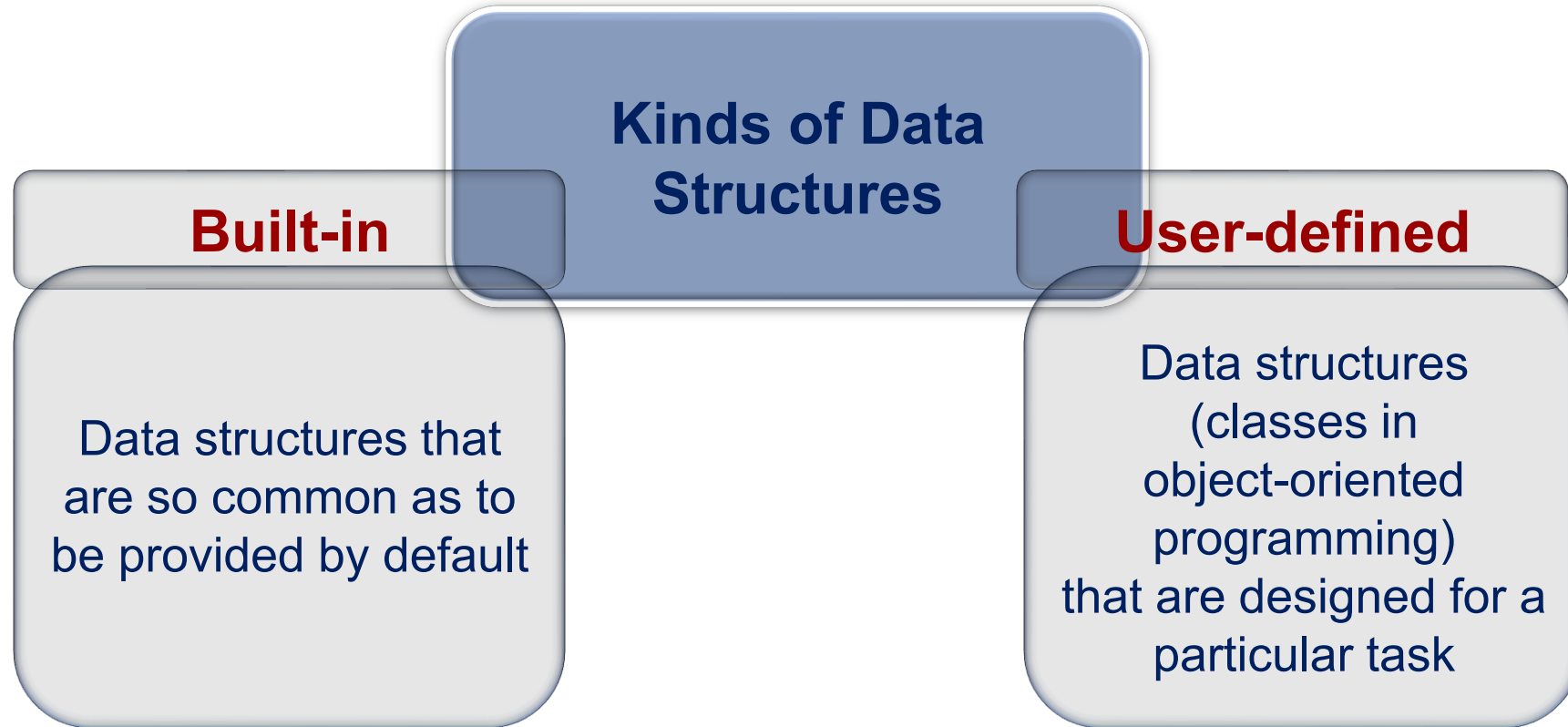
What is a Composite Type?



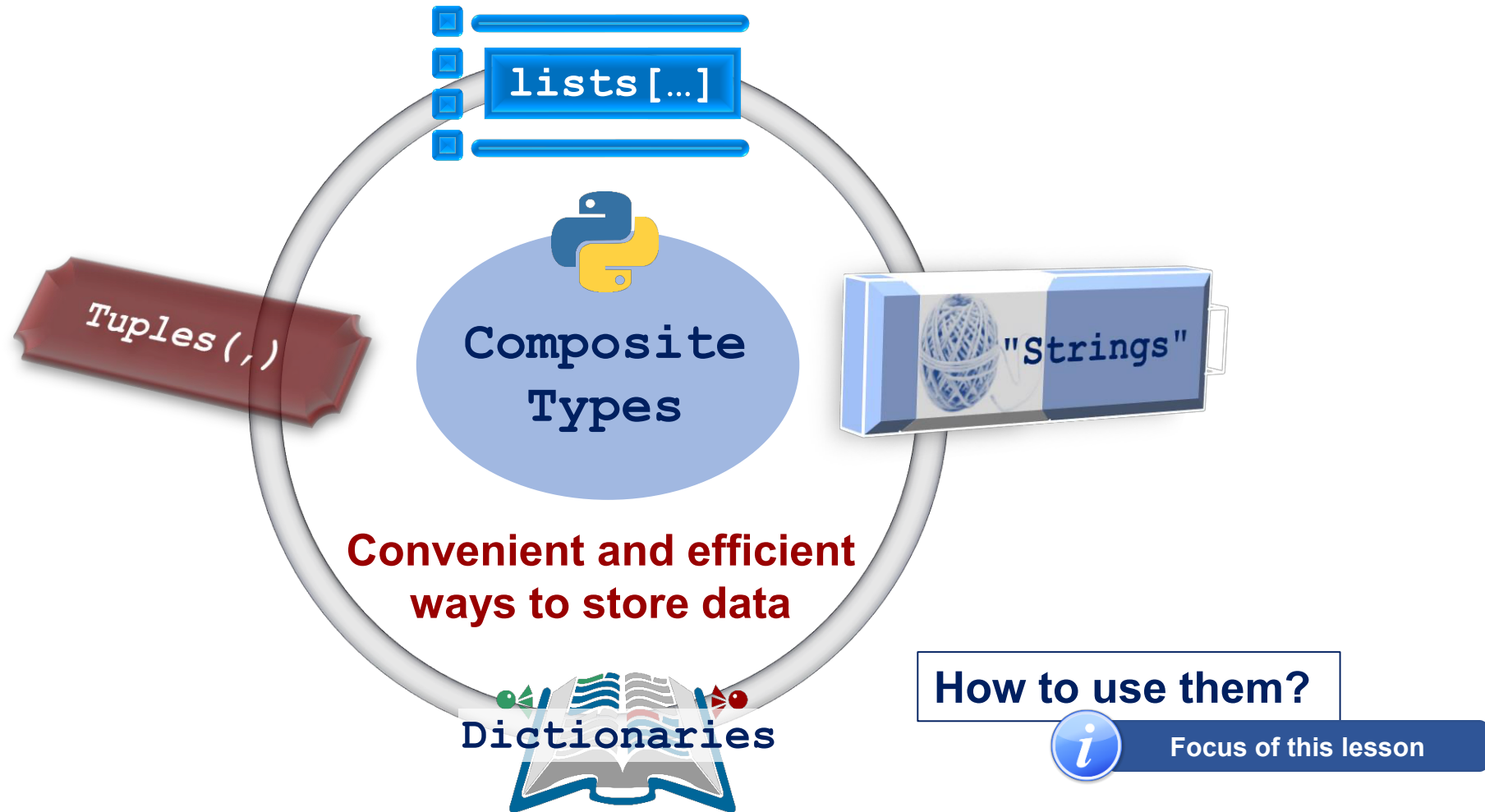
- A **data type**, which is constructed (composed) **using primitive** and **other composite types**.
- A **new data type** made from existing ones.

- **Particular ways of storing data** to make some operations easier or more efficient
 - They are **tuned** for certain tasks, and they are often associated with algorithms
- Different data structures have different characteristics
 - One suited to solving a **certain problem** may not be suited for another problem





Composite Types in Python





Python List is an **ordered sequence of items**.



Recall

We have already covered a type of sequence: **Strings**

- A **string** is a **sequence of characters**.

Creating a List

- As with all data structures, lists have a **constructor**.
- **Constructors** have the same name as the data structures.

```
l = list()
```



Creates an empty list

```
l = list(arg)
```



Takes an **iterable** data structure as an argument and add each item of **arg** to the constructed list **l**

- **Shortcut:** use of **square brackets []** to indicate explicit items.



```
l = [...]
```

Creating a List: Example

```
aList = list('abc')
```

```
aList ⇒ ['a', 'b', 'c']
```

```
newList = [1, 3.14159, 'a', True]
```

Lists: Similarities with Strings

- **concatenate**: `+` (only for lists – not `string + list`)
- **repeat**: `*`
- **indexing**: the `[ ]` operator), e.g., `lst[3]`  4th item in the list
- **slicing**: `[ : ]`
- **membership**: the `in` operator
- **length**: the `len()` function

Lists: Differences with Strings

- Lists can contain **a mixture of python objects (types)**; strings can **only hold characters**.

E.g. `l = [1, 'bill', 1.2345, True]`

- Lists are **mutable**; their values can be changed, while strings are **immutable**.
- Lists are designated with `[ ]`, with elements separated by commas; strings use `""`.

List Structure

```
myList = [1, 'a', 3.14159, True]
```

myList	1	'a'	3.14159	True
Index Forward	0	1	2	3
Index Backward	-4	-3	-2	-1

`myList[1]` → 'a'

`myList[:3]` → [1, 'a', 3.14159]

[] ? Indexing on Lists

[] means a list and it is also used to retrieve index.

`['a', 'b', 'c'][1]` → `'b'`

`[0, 1, 2][0]` → `0`

`[0][0]` → `0`

Content is important!

Index is always at the end of the expression and is preceded by something (variable, **sequence**).

Lists of Lists

```
myLst = ['a', [1, 2, 3], 'a']
```



What is the second element of the list?

```
myLst[1][0]    #apply from left to right
```

```
myLst[1] → [1, 2, 3]
```

```
[1, 2, 3][0] → 1
```

```
[1, [2, [3, 4]], 5][1][1][0] → ?
```

Operators

+

e.g. `[1, 2, 3] + [4]` $\rightarrow$ `[1, 2, 3, 4]`

*

e.g. `[1, 2, 3] * 2` $\rightarrow$ `[1, 2, 3, 1, 2, 3]`

`in`

e.g. `1 in [1, 2, 3]` $\rightarrow$ `True`

List Functions

len(list) Number of elements in list (top level)

e.g. `len([1, [1, 2], 3])` → 3

min(list) Minimum element in the list

max(list) Maximum element in the list

sum(list) Sum of the elements, numeric only

Iterate on the List



```
for element in [1, [1, 2], 'a', True]:  
    print(element)
```

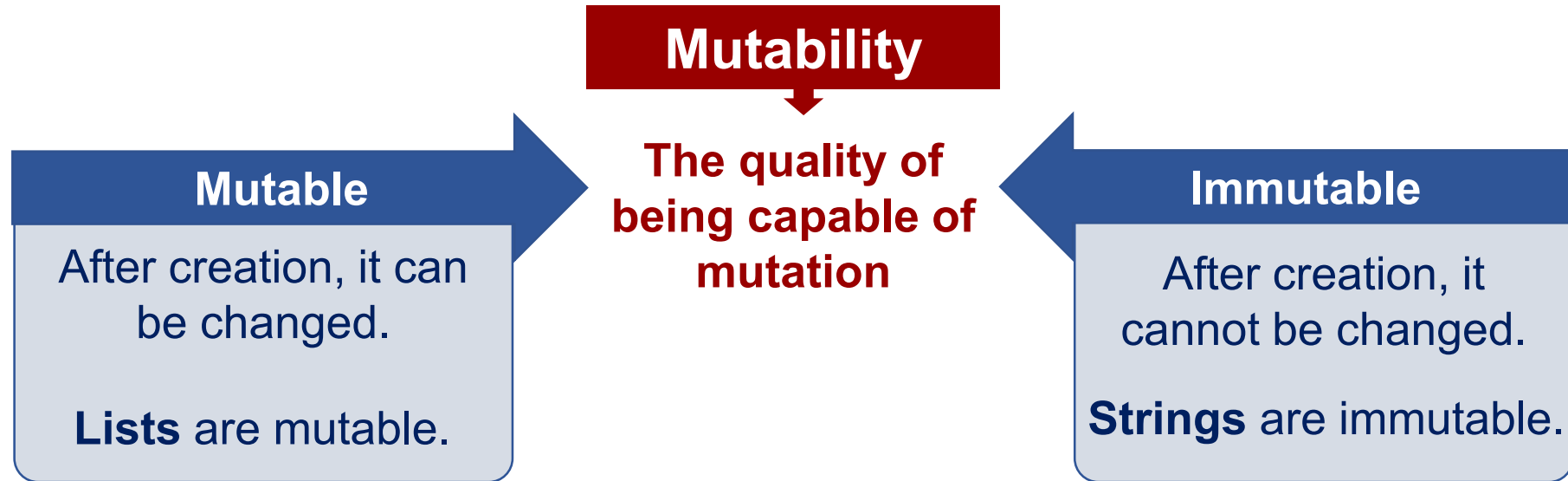


What do you think is the print output?

Answer:



1
[1, 2]
'a'
True



Immutable (Strings): Examples

```
myStr = 'abc'
```

```
myStr[0] = 'z' #not possible
```

```
newStr= myStr.replace('a', 'z') #make a new string
```

The object's contents can be changed.



```
myLst = [1, 2, 3]  
myLst[0] = 127  
print(myLst)
```



What do you think is the output?

Answer: ➡ **[127, 2, 3]**

A list is mutable and can be changed:

```
myList[0] = 'a'           #index assignment
myList.append(e)          // e: element to append
myList.extend(L)          // L: a list
myList.pop(i)             // i: index (default: -1)
myList.insert(i,e)
myList.remove(e)
myList.sort()
myList.reverse()
```

List Methods: Example

<code>myList = [1,3]</code>	<code>[1, 3]</code>
<code>myList[0] = 'a'</code>	<code>['a', 3]</code>
<code>myList.append(2)</code>	<code>['a', 3, 2]</code>
<code>lst = [6,5]</code>	
<code>myList.extend(lst)</code>	<code>['a', 3, 2, 6, 5]</code>
<code>myList.extend(5)</code>	ERROR!
<code>element = myList.pop()</code>	<code>['a', 3, 2, 6]</code>
<code>print(element)</code>	5
<code>myList.append([8,9])</code>	<code>['a', 3, 2, 6, [8,9]]</code>

List Methods: Example (Cont'd)

```
myList.insert(0, 'b')  
myList.insert(-1, 'b')  
myList.insert(10, 'c')  
myList.remove('b')
```

```
['a', 3, 2, 6]  
['b', 'a', 3, 2, 6]  
['b', 'a', 3, 2, 'b', 6]  
['b', 'a', 3, 2, 'b', 6, 'c']  
['a', 3, 2, 'b', 6, 'c']
```

```
myList.sort()
```

```
TypeError!!
```

```
myList.remove('b')  
myList.remove('a')  
myList.remove('c')
```

```
['a', 3, 2, 6, 'c']  
[3, 2, 6, 'c']  
[3, 2, 6]
```

```
myList.remove('d')
```

```
ValueError!!
```

```
myList.sort()
```

```
[2, 3, 6]
```

```
myList.reverse()
```

```
[6, 3, 2]
```

Return Values

- When compared to string methods, most of these list methods **do not** return a value.
- This is because **lists are mutable** so the **methods modify the list directly**; there is no need to return a new list.



Remember the python
standard is your friend!



Warning about Results

```
myLst = [4, 7, 1, 2]
```

```
myLst = myLst.sort()
```

```
myLst ⇒ None      #what happened?
```



What is the return value of `myLst.sort()`?

Be careful of
what you are
assigning!



WARNING

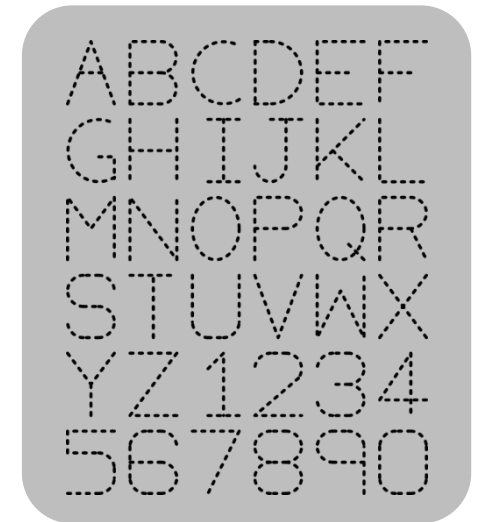
String Method: `split()`

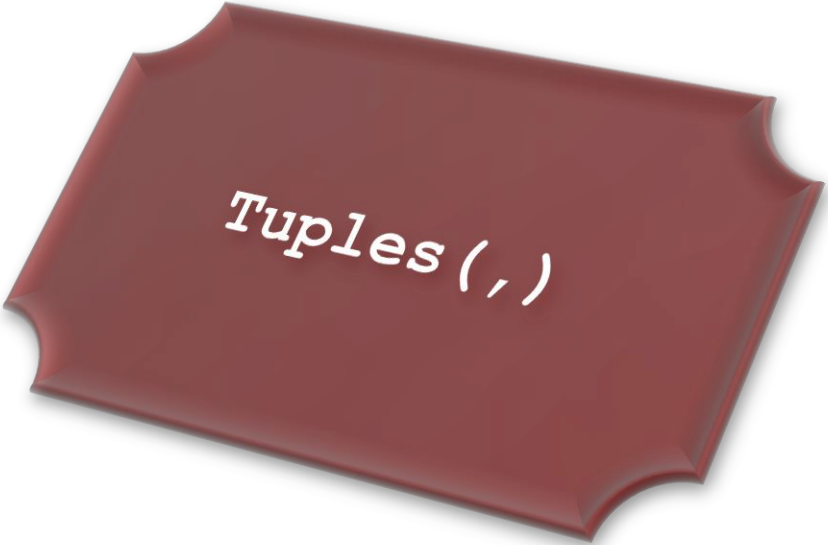
- The string method `split()` generates a sequence of characters by splitting the string at certain split-characters.
 - Default split-character: **white space**.
- The string method, `split()`, returns a list.

```
splitLst = 'this is a test'.split()  
print(splitLst) ➔ ['this', 'is', 'a', 'test']
```

- Only lists have a **built-in sorting method**.
- Thus, data could be **converted to a list** if it needs sorting.

```
myLst = list('xyzabc') #iterable to constructor  
myLst ➡ ['x', 'y', 'z', 'a', 'b', 'c']  
myLst.sort() ➡ ['a', 'b', 'c', 'x', 'y', 'z']  
# convert back to a string  
sortStr = ''.join(myLst) ➡ 'abcxyz'
```





Tuples (,)

Tuples (,)

Tuples are **immutable** lists.

Why Immutable Lists?

- Provides a data structure with some integrity and some permanency
- To avoid accidentally changing one

They are designated with **(,)**.

Example:

```
myTuple = (1, 'a', 3.14, True)
```

Lists vs. Tuples

Everything that works for a list works for a tuple **except** methods that modify the tuple.

What works?

- indexing
- slicing
- `len()`
- `print()`

What doesn't work?

Mutable methods

- `append()`
- `extend()`
- `remove()` , etc.

Commas Create Tuples

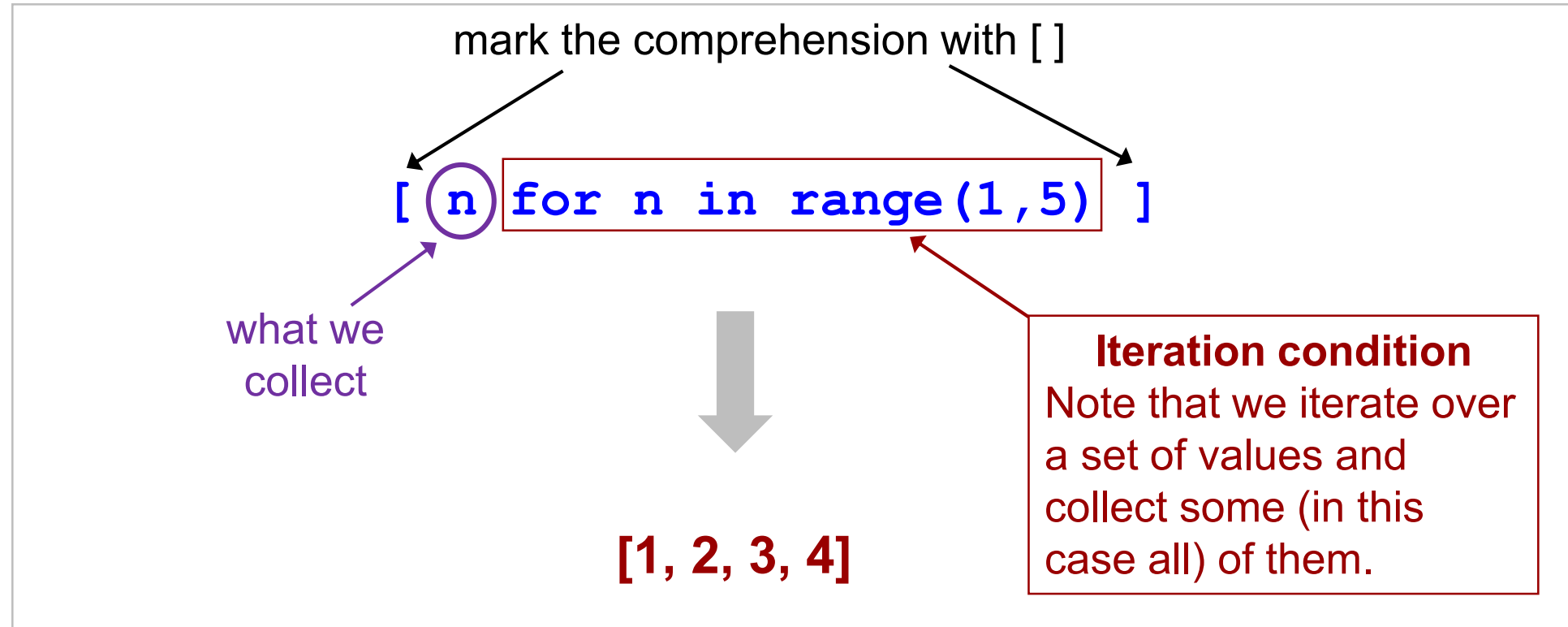
For tuples:

- **comma** can be thought of as the **operator** that makes a tuple
- while the **round bracket ()** simply acts as a **grouping**

```
myTuple = 1,2      # creates (1,2)
myTuple = (1,)      # creates (1)
myTuple = (1)       # creates 1 not (1)
myTuple = 1,        # creates (1)
```


List Comprehension

List comprehension: syntactic structure for concise construction of lists



Other Examples

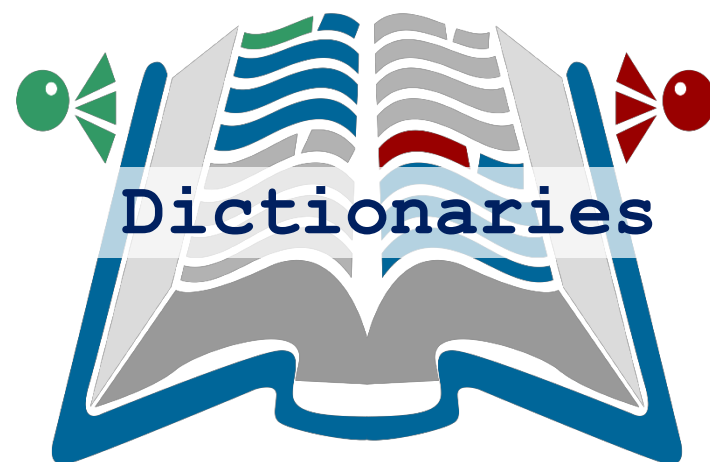
`[n**2 for n in range(1,6)]`  `[1, 4, 9, 16, 25]`

`[x + y for x in range(1,5) for y in range (1,4)]`  `?`

It is as if we had done the following:

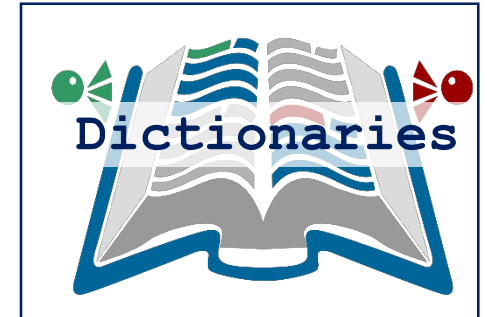
```
myList = [ ]  
for x in range (1,5):  
    for y in range (1,4):  
        myList.append(x+y)
```

`[c for c in "Hi There Mom" if c.isupper()]`  `['H', 'T', 'M']`



What is Dictionary?

- In data structure terms, a **dictionary** is better termed as an **associative array**, or **associative list**, or a **map**.
- You can think of it as a **list of pairs**.
 - The **key**, which is the **first element** of the pair, is used to retrieve the **second element**, which is the **value**.
- Thus, we map a key to a value.



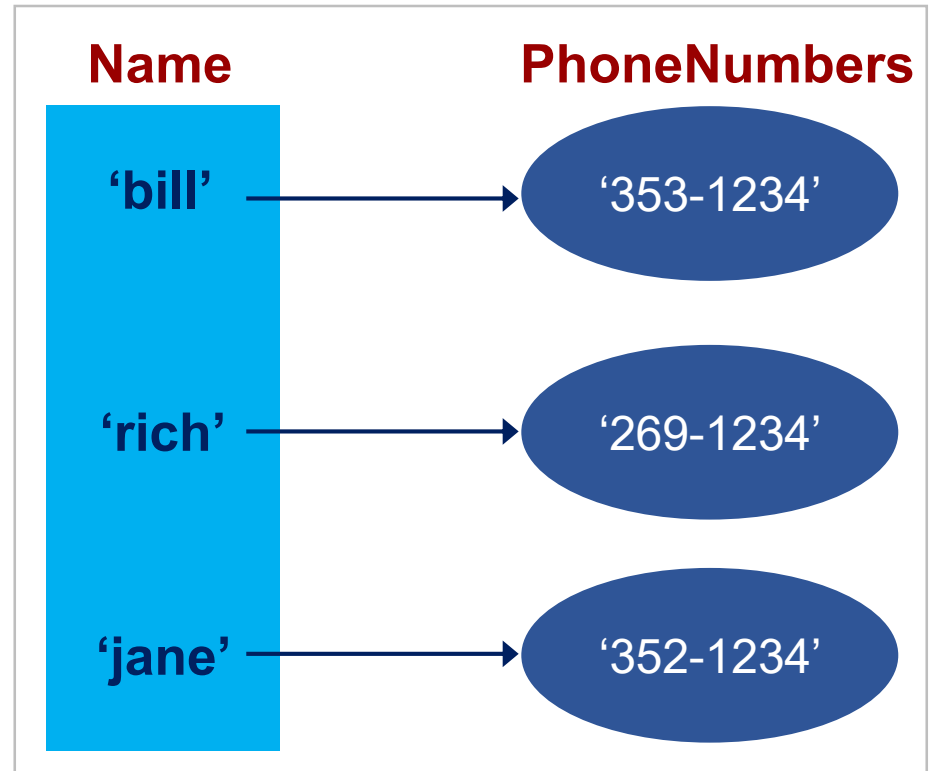
Key:Value

- The **key** acts as a “lookup” to find the associated value.
- Just like a dictionary, you look up a word by its spelling to find the associated definition.
- A dictionary can be searched to locate the value associated with a key.

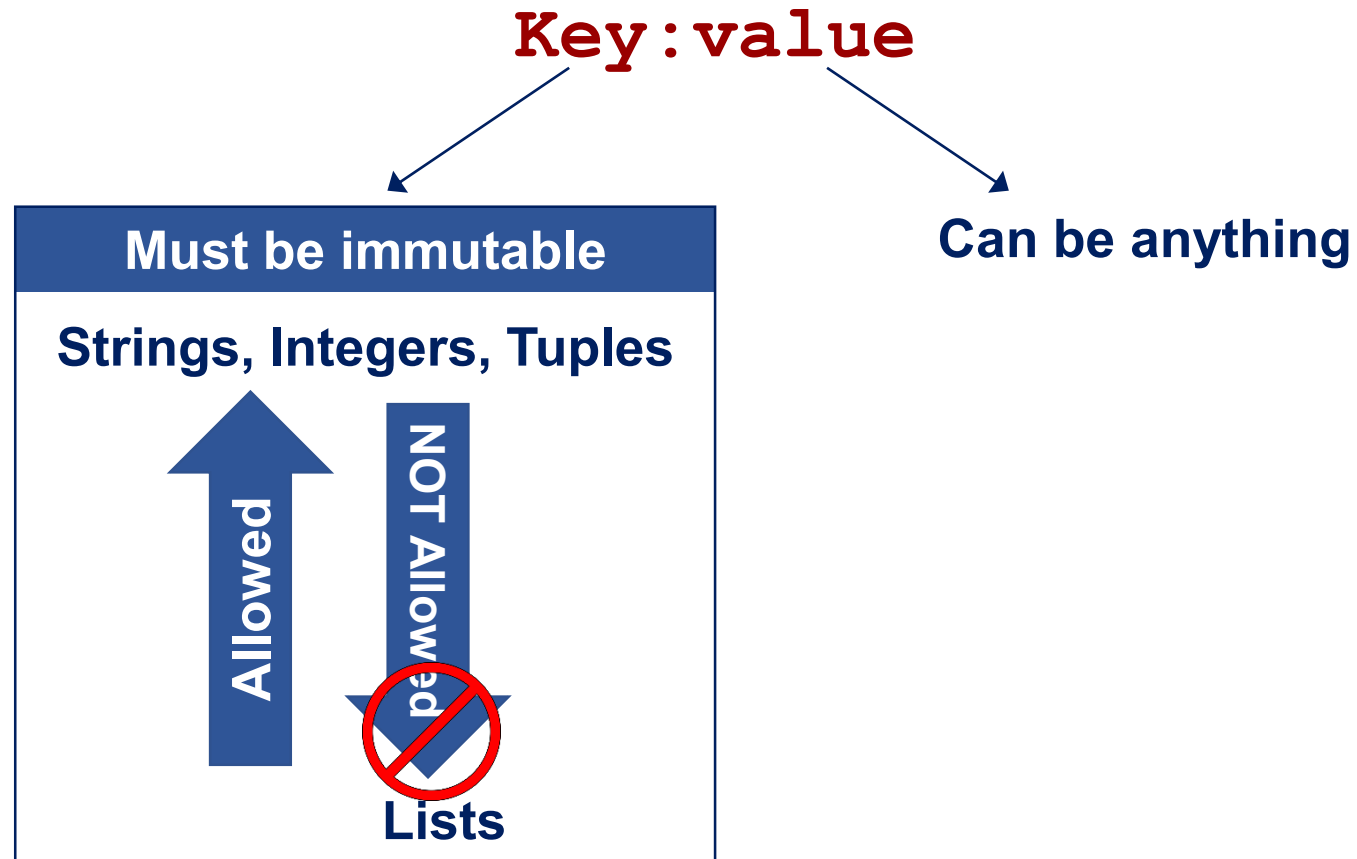
{ } **marker**: used to create a dictionary

: **marker**: used to create **key:value** pairs

```
contacts = {'bill': '353-1234',  
            'rich': '269-1234',  
            'jane': '352-1234'}  
  
print(contacts) ➔ {'jane': '352-1234',  
                   'bill': '353-1234',  
                   'rich': '269-1234'}
```



What are Keys and Values?



Collection vs. Sequence

Dictionaries are **collections** but they are **not sequences** like lists, strings, or tuples.

- There is **no order** to the elements of a dictionary.
- In fact, the order (for example, when printed) might change as elements are added or deleted.



So, how do you access dictionary elements?

Access to Dictionary

Access requires `[]` and the **key** is the index.

`myDict = {}`  **an empty dictionary**

`myDict['bill'] = 25`  **add the pair 'bill':25**

`print(myDict['bill'])`  **print 25**

`del myDic['bill']`  **remove the pair 'bill':25**

Dictionaries are Mutable

Like lists, dictionaries are mutable.

- You can change the object via various operations, such as index assignment.

```
myDict = {'bill':3, 'rich':10}

print(myDict['bill'])           # prints 3
myDict['bill'] = 100           # change value
print(myDict['bill'])           # prints 100

del myDict['rich']              # remove 'rich':10
del myDict['rich']              # KeyError
```

Dictionary Operations

Like others, dictionaries respond to these:

`len(myDict)` → **number** of **key:value pairs** in the dictionary

`element in myDict` → boolean; is **element** a **key** in the dictionary?

`for key in myDict` → iterate through the **keys** of a dictionary

Other Methods and Operations

`myDict.items()` → return all the **key:value** pairs

`myDict.keys()` → return all the keys

`myDict.values()` → return all the values

`myDict.clear()` → empty the dictionary

`myDict.update(yourDict)` → for each key in **yourDict**, update **myDict** with that **key:value** pair

Iterating on a Dictionary

```
for key in myDict:  
    print(key) ➡ prints all the keys
```

```
for key,value in myDict.items():  
    print(key, value) ➡ prints all the key:value pairs
```

```
for value in myDict.values():  
    print(value) ➡ prints all the values
```

In this lesson, we have learnt:

- The concept of composite types
- Built-in composite types in the Python programming language:
 - List
 - Tuple
 - Dictionary

References for Images

No.	Slide No.	Image	Reference
1	5		Gabovitch, I. (2014). AV Out In HDMI In Jack Plug Red White Yellow Audio and Video Mixer Backside [Online Image]. Retrieved May 17, 2018 from https://www.flickr.com/photos/qubodup/12248078123 .
2	6		Python Logo [Online Image]. Retrieved April 24, 2018 from https://pixabay.com/en/language-logo-python-2024210/ .
3	6, 8, 37, 38		By Ephemeron - Own work, based on File:Dynamic Dictionary Logo.png, CC BY-SA 3.0, retrieved May 18, 2018 from https://commons.wikimedia.org/w/index.php?curid=7361291 .
4	6, 8		String [Online Image]. Retrieved April 24, 2018 https://pixabay.com/en/string-twine-ball-twined-isolated-314346/ .
5	10		Search [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/database-search-database-search-icon-2797375/ .

References for Images

No.	Slide No.	Image	Reference
6	17, 20, 22, 42		Question problem [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/question-problem-think-thinking-622164/ .
7	21		Survey icon [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/survey-icon-survey-icon-2316468/ .
8	26		Smiley 11 [Online Image]. Retrieved April 18, 2018 from http://www.publicdomainfiles.com/show_file.php?id=13545100814144 .
9	27		By Unknown - From the Open Clip Art Gallery - http://openclipart.org/ , CC0, retrieved May 16, 2018 from https://commons.wikimedia.org/w/index.php?curid=1849852 .
10	29		Alphabet [Online Image]. Retrieved May 17, 2018 from https://pixabay.com/en/alphabet-letters-numbers-digits-40515/ .



Strings in Python



At the end of this lesson, you should be able to:

- Describe the terminology, 'string', as used in Python
- Define strings in Python
- Access characters in a Python string
- Slice a Python string to get substrings
- Use common string functions and methods in Python

Topic Outline



Definition of a String



Accessing Strings



Strings Operations



Functions and Methods for String

What is a String?

A string is a sequence of characters.



Notionally, a character is a letter or a symbol.



- A string is indicated using **single quote** ('...') or **double quotes** ("...")
 - For example, 'abc' or "abc"
- The **sequence** of characters is important and is maintained

Defining a String (Cont'd)

- Using a single or double quotes is fine



Example:

`s = "a string"`

or

`s = 'a string'`

- Don't use both



Example:

`s = "a string'` → **WRONG**



Syntax Error

How to put an apostrophe in a string, e.g., Mike's book?

- `s = "Mike's book"` → Use different quotes
- `s = 'Mike\'s book'` → Use an escape character

- Characters in a string are in a **sequence**.
- We can identify each character with an unique **index** (a position in the sequence).
- We can index a character from either end of the sequence.
 - Non-negative values: counting from **left**, **starting at 0**
 - Negative values: counting from **right**, **starting at -1**

Index (Cont'd)

String = "Hello World"

Characters	H	e	l	l	o		W	o	r	l	d
	0	1	2	3	4	5	6	7	8	9	10
Indices									...	-2	-1

*Indices 10 and -1 point to the same location: **d***

Accessing One Element

We can use `[]` to access particular characters in a string.

```
myStr = "Hello World"
```

```
x = myStr[1]
```

```
print(x)
```



#will print **e**

```
print(myStr)
```



? #will print **Hello World**

```
print(myStr[-2])
```



? #will print **l**

```
print(myStr[11])
```



?  **Error**



Characters

	H	e	l	l	o		W	o	r	l	d
Indices	0	1	2	3	4	5	6	7	8	9	10
									...	-2	-1

Slicing: Parts of a String

We can also select a part of the string.

Syntax: [**start** : **finish**]

↑
the index of the end of a subsequence (*not included*)

↓
the index of the start of a subsequence

*By default, these indices, (**start** and **finish**), will point to the **beginning** and **end** of the string, respectively.*

Slicing: Parts of a String (Cont'd)

The start index is inclusive while the **finish index** is the **one after** the subsequence.

`myStr = "Hello World"`

`myStr[1:4]`  ?

Characters

H	e	l	l	o		W	o	r	l	d
0	1	2	3	4	5	6	7	8	9	10
								...	-2	-1

Slicing: Parts of the String (Cont'd)

Example:

```
myStr = "Hello World"
print(myStr[1:6])
print(myStr[1:2])
print(myStr[-7:-1])
print(myStr[-3:-5])
print(myStr[:6])
print(myStr[5:])
```

Output

'ello '

'e'

'o Worl'

'' (empty)

'Hello '

' World'

Characters

Indices

H	e	l	l	o		W	o	r	l	d
0	1	2	3	4	5	6	7	8	9	10
-12	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Slicing: Parts of the String (Cont'd)

One More Example:

`myStr[3:-2]`

Characters	H	e	l	l	o		W	o	r	l	d
Indices	0	1	2	3	4	5	6	7	8	9	10
										...	-2
											-1

We can also specify a third argument.

Syntax: [**start** : **finish** : **step**]



specifies the step size to jump along the sequence

Defaults:

- **start**: beginning
- **finish**: end
- **step**: 1

Extended Slicing (Cont'd)

`myStr[::2]`  **'HloWr d'**

Characters	H	e	l	l	o		W	o	r	l	d
Indices	0	1	2	3	4	5	6	7	8	9	10



Extended Slicing: Common Patterns

Copying a String

```
aString = "String to copy"
```

```
newStr = aString[:]
```

```
newStr = ''.join(aString)
```

Reversing a String

```
aString = "Madam I'm Adam"
```

```
revString = aString[::-1] → "maDA m' I madaM"
```

```
bString = aString[0:14:-1] → ""
```

Basic Operations

```
opStr = "Basic"
```

Length of a string: `len()`

e.g. `len(opStr)` → 5

Concatenate strings: `+`

e.g. `opStr + "operations"` → 'Basic operations'

Repeat String: `*`

e.g. `opStr * 3` → 'BasicBasicBasic'

Two common systems for representing characters:

- **ASCII** (older)
- **Unicode** (modern, more characters)

Tables indicate mapping of characters to ASCII or Unicode.

ASCII vs. Unicode



ASCII

- Uses 8 bits to store a character
- $2^8 = 256$ different characters

Unicode

- An extension of ASCII
- Able to include more characters
- Uses 16 bits to store a character
- $2^{16} = 65,536$ characters

- The Unicode space is divided into 17 planes.
- Each plane contains 65,536 code points (16-bit).

Total of:
1,114,112 characters,
96,000 used.

ACSII Table (Partial)

Dec	Hx	Oct	Html	Char	Dec	Hx	Oct	Html	Char	Dec	Hx	Oct	Html	Char	Dec	Hx	Oct	Html	Char	Dec	Hx	Oct	Html	Char
32	20	040	 	Space	52	34	064	4	4	72	48	110	H	H	92	5C	134	\	\	112	70	160	p	p
33	21	041	!	!	53	35	065	5	5	73	49	111	I	I	93	5D	135	]	]	113	71	161	q	q
34	22	042	"	"	54	36	066	6	6	74	4A	112	J	J	94	5E	136	^	^	114	72	162	r	r
35	23	043	#	#	55	37	067	7	7	75	4B	113	K	K	95	5F	137	_	_	115	73	163	s	s
36	24	044	$	\$	56	38	070	8	8	76	4C	114	L	L	96	60	140	`	`	116	74	164	t	t
37	25	045	%	%	57	39	071	9	9	77	4D	115	M	M	97	61	141	a	a	117	75	165	u	u
38	26	046	&	&	58	3A	072	:	:	78	4E	116	N	N	98	62	142	b	b	118	76	166	v	v
39	27	047	'	'	59	3B	073	;	;	79	4F	117	O	O	99	63	143	c	c	119	77	167	w	w
40	28	050	(	(	60	3C	074	<	<	80	50	120	P	P	100	64	144	d	d	120	78	170	x	x
41	29	051	)	)	61	3D	075	=	=	81	51	121	Q	Q	101	65	145	e	e	121	79	171	y	y
42	2A	052	*	*	62	3E	076	>	>	82	52	122	R	R	102	66	146	f	f	122	7A	172	z	z
43	2B	053	+	+	63	3F	077	?	?	83	53	123	S	S	103	67	147	g	g	123	7B	173	{	{
44	2C	054	,	,	64	40	100	@	@	84	54	124	T	T	104	68	150	h	h	124	7C	174	|	
45	2D	055	-	-	65	41	101	A	A	85	55	125	U	U	105	69	151	i	i	125	7D	175	}	}
46	2E	056	.	.	66	42	102	B	B	86	56	126	V	V	106	6A	152	j	j	126	7E	176	~	~
47	2F	057	/	/	67	43	103	C	C	87	57	127	W	W	107	6B	153	k	k	127	7F	177		DEL
48	30	060	0	0	68	44	104	D	D	88	58	130	X	X	108	6C	154	l	l					
49	31	061	1	1	69	45	105	E	E	89	59	131	Y	Y	109	6D	155	m	m					
50	32	062	2	2	70	46	106	F	F	90	5A	132	Z	Z	110	6E	156	n	n					
51	33	063	3	3	71	47	107	G	G	91	5B	133	[	[	111	6F	157	o	o					

Source: www.lookupTables.com

Getting the Code

`ord()` takes a **character** as input and returns the Unicode of the character (for standard symbols this is the same as in ASCII).

e.g. `ord('a')` → **97**

`chr()` takes an ASCII/UTF-8 code and returns the corresponding character.

e.g. `chr(97)` → **'a'**

For encryption: `code = ord('a')`
`chr (code + 1)` → **'b'**

Comparing Two Characters

The code is used for comparison.

'a' == 'a' → true

'a' < 'b' → true

'1' < '9' → true

'a' < 'B' → false!



Is one string contained in another?

- Operator: **in**
- **a in b**: True if string **a** is contained in string **b**

```
myStr = "abcdefg"
```

```
'c' in myStr    → true
```

```
'cde' in myStr  → true
```

```
'cef' in myStr  → false
```

```
myStr in myStr → true
```

Strings are Immutable

- Strings are immutable, i.e., you cannot change one once you make it.

- `aStr = 'spam'`

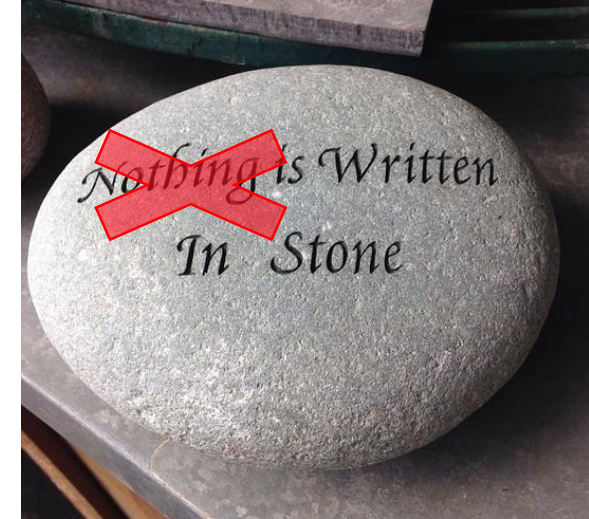
- `aStr[1] = 1` →  Error

- However, you can use it to make another string (copy it, slice it, etc.).

- `newStr = aStr[:1] + 'l' + aStr[2:]`

- `newStr` → **'slam'**

- `aStr` → **'spam'**



Functions and Methods for Strings

- A **function** is a **piece of code** that performs some operation.
 - The details are hidden (encapsulated), only it's interface exposed.
 - It is a way to arrange a program to make it **easier to understand**.
- A function has arguments as inputs and may return one output.

String Functions

A String Function: `len()`



Recall Length of a string: `len()`

e.g. `len('test string')` → **11**
(not 10)

- Input: a string
- Output: an integer indicating the length of the string

- A method is a variation on a function.
 - It represents a program.
 - It has input arguments and output.
- Unlike a function, it is applied in the context of a particular object.
 - This is indicated by the **dot notation** invocation.
- Each string is itself an object.

String Method: `upper()`

`upper()` is a string method.

It will output a new string, which is the same as the string on which it was called, except all letters will now be in upper case.

```
myStr = "shouting!"  
myStr.upper() → 'SHOUTING!'
```

- Object: `myStr`
- Method: `upper()`
- Method call: `myStr.upper()`

Syntax: `object.method()`

- We say, **object** is calling the method **method**.

Different objects have different methods.



How do we find out all the methods available for strings?

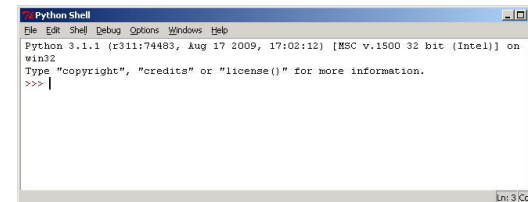
Use Reference

- **Python online**



<http://docs.python.org/release/3.2.3/library/stdtypes.html#string-methods>

- Integrated **D**evelopment **E**nvironment (**IDE**) such as **IDLE**



String Method: `find()`

`find()` is another string method.

```
myStr = "Find in a string"  
myStr.find('d') → 3
```

→ 'd' is called an **argument** of the method.

- Input: a **single character**
- Output: the **index** of the character (first seen from left to right)
- If the character is not found, **-1** is returned

String Method: `join()`

`join()` is another string method.

```
str1 = "abcd"  
str2 = "1234"  
str2.join(str1) ➡ 'a1234b1234c1234d'  
str1.join(str2) ➡ '1abcd2abcd3abcd4'
```

Syntax: `base.join(target)`

- Input: the **target string** to be joined
- Output: the new string where the **base** joins the **target**

Methods can be chained together.

Perform first operation, yielding an object.



Use the resulting object for the next method.

```
myStr = "Python Rules!"
```

```
myStr.upper()
```



'PYTHON RULES!'

```
myStr.upper().find('H')
```



3

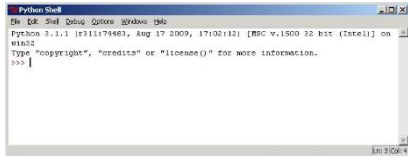
In this lesson, we have learnt:

- How to define a string in Python
- How to access characters in a Python string based on indices
- How to slice a Python string
- Common functions and methods for strings in Python

References for Images

No.	Slide No.	Image	Reference
1	5		String [Online Image]. Retrieved May 9, 2018 from https://www.flickr.com/photos/epublicist/8718123610 .
2	6		Search [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/database-search-database-search-icon-2797375/ .
3	7, 10		By PKua - Own work, Public Domain, retrieved May 9, 2018 from https://commons.wikimedia.org/w/index.php?curid=3929297 .
4	7, 13, 14		Survey icon [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/survey-icon-survey-icon-2316468/ .
5	10		Python Logo [Online Image]. Retrieved April 24, 2018 from https://pixabay.com/en/language-logo-python-2024210/ .

References for Images

No.	Slide No.	Image	Reference
6	24, 31		Question problem [Online Image]. Retrieved April 24, 2018 from https://pixabay.com/en/question-problem-think-thinking-622164/ .
7	25		Written in stone [Online Image]. Retrieved May 9, 2018 from https://pixabay.com/en/nothing-is-written-in-stone-rock-527756/ .
8	31		World Wide Web [Online Image]. Retrieved May 9, 2018 from https://pixabay.com/en/world-wide-web-internet-computer-24958/ .
9	31		CC BY-SA 3.0, retrieved May 9, 2018 from https://commons.wikimedia.org/w/index.php?curid=11887635 .